

VISÃO COMPUTACIONAL

Fundamentos de Imagens Digitais e Histograma

Prof. Dr. José Jairo de S. e Silva

Aula 01

Fundamentos de Imagens Digitais e Histograma

PLANO DA DISCIPLINA



Ementa

- Fundamentos de imagens digitais
- Representação de imagens
- Histograma
- Realce de imagens no domínio do espaço
- Realce de imagens no domínio da frequência
- Segmentação
- Morfologia Matemática
- Técnicas de aprendizado de máquina
- Redes neurais artificiais
- Redes neurais convolucionais
- CNNs para classificação de objetos
- CNNs para detecção de objetos

🎯 Objetivos de Aprendizagem

1. Compreender representação de imagens digitais, histograma e aplicar equalização de histograma
2. Compreender realce no domínio do espaço e da frequência e aplicar técnicas de realce
3. Compreender segmentação e aplicar detecção de linhas, pontos e bordas
4. Compreender e aplicar operações morfológicas em imagens digitais
5. Compreender e implementar redes neurais multicamadas para classificação de imagens
6. Compreender e implementar redes neurais convolucionais para classificação de imagens
7. Aplicar CNNs para classificação de objetos
8. Aplicar CNNs para detecção de objetos



Metodologia

Como aprenderemos:

- Aulas expositivas dialogadas
- Ensino baseado em problemas
- Implementação de aplicações práticas
- Atividades no Blackboard

Ferramentas utilizadas:

- [Python](#) (linguagem principal)
- [OpenCV](#) (processamento de imagens)
- [NumPy](#) (operações matriciais)
- [Matplotlib](#) (visualização)
- [PyTorch / Keras](#) (redes neurais)

Sistema de Avaliação

Composição da nota:

Avaliação	Valor	Período
A1	0,0 a 5,0	1º Bimestre
A2	0,0 a 5,0	2º Bimestre
$NF = A1 + A2$ 0,0 a 10,0 —		

Aprovado: $NF \geq 6,0$ e frequência $\geq 75\%$

Instrumentos utilizados:

- Avaliação teórica
- Entrega de trabalhos (Blackboard)
- Atividades/exercícios no Blackboard
- Atividades realizadas em aula
- Projeto disciplinar ou interdisciplinar

Avaliação Final Substitutiva (AF):

Substitui a **menor nota** entre A1 e A2, para alunos com $NF < 6,0$ e frequência $\geq 75\%$.

Bibliografia

Básica

- GONZALEZ, R. C.; WOODS, R. E. [Processamento de imagens digitais](#). São Paulo: Blucher, 2000.
- JAIN, A. K. [Fundamentals of digital image processing](#). Englewood Cliffs: Prentice-Hall, 1989.
- BAB-HADIASHAR, A.; SUTER, D. [Data segmentation and model selection for computer vision](#). New York: Springer, 2000.

Complementar

- FILHO, O. M.; NETO, H. V. [Processamento Digital de Imagens](#). Brasport, 1999.
- BARELLI, F. [Introdução à Visão Computacional](#). Casa do Código, 2019.
- CONCI, A.; AZEVEDO, E.; LETA, F. R. [Computação gráfica: teoria e prática](#). Elsevier, 2008.
- HETEM JUNIOR, A. [Computação gráfica](#). LTC, 2006.

Livro de Referência da Disciplina



Introdução à Visão Computacional

Uma abordagem prática com Python e OpenCV

Autor: Felipe Barelli

Editora: Casa do Código, 2019

Este livro será nossa principal referência prática ao longo da disciplina. Os exemplos de código, imagens e exercícios foram desenvolvidos com **Python** e **OpenCV**, as mesmas ferramentas que utilizaremos nas aulas.

OBJETIVOS DA AULA

🎯 Objetivo de Aprendizagem

Ao final desta aula, o estudante será capaz de:

Compreender o conceito de **representação de imagens digitais**, **histograma** e aplicar **técnicas de equalização de histograma**.

Conteúdos:

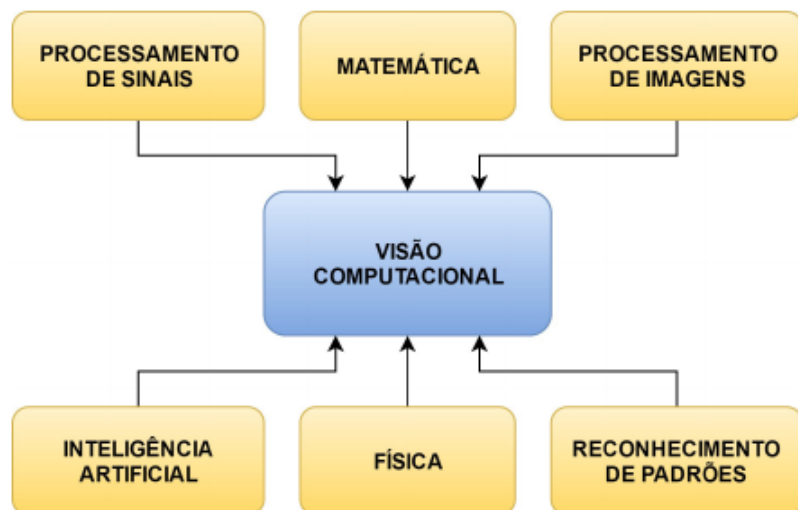
- Fundamentos de imagens digitais
- Representação de imagens (pixel, resolução, quantização)
- Histograma de cores
- Técnicas de equalização de histograma

O QUE É VISÃO COMPUTACIONAL?

Definição

A **Visão Computacional** é a ciência que estuda e desenvolve tecnologias que permitem que **máquinas enxerguem** e extraíam características do meio através de imagens capturadas por diferentes tipos de sensores e dispositivos.

Campos relacionados:



Dispositivos de captura:

- Câmeras digitais
- Sensores CCD/CMOS
- Raio-X
- Ultrassom
- Infravermelho

Aplicações da Visão Computacional

Indústria e Automação

- Robôs industriais para montagem
- Controle de qualidade
- Inspeção de produtos

Segurança

- Sistemas de monitoramento
- Detecção de pessoas
- Reconhecimento facial

Saúde

- Diagnóstico por imagem
- Tomografia computadorizada
- Ressonância magnética
- Detecção de tumores

Veículos Autônomos

- Detecção de placas
- Identificação de pedestres
- Rastreamento de faixas

Dispositivos para Aquisição de Imagem

Sensores e dispositivos são utilizados para **capturar e digitalizar imagens** do ambiente. Alguns deles conseguem registrar imagens mesmo onde **não há luz visível**, como o Raio-X e o Ultrassom.



Figura 1.1 — Barelli (2019)

Fluxo de um Sistema de Visão Computacional

Etapa	Descrição
1. Aquisição	Captura e digitalização por sensores
2. Pré-processamento	Realce de bordas, tratamento de ruídos
3. Segmentação	Separação dos objetos de interesse
4. Extração	Obtenção de atributos (área, diâmetro, etc.)
5. Reconhecimento	Classificação dos objetos em categorias



Figura 1.5 — Barelli (2019)

FUNDAMENTOS DE IMAGENS DIGITAIS

O que é uma Imagem Digital?

Uma imagem digital é uma **matriz bidimensional** onde cada elemento representa um **pixel** (*picture element*).

$$\text{Imagem Digital} = \text{Matriz}[L \times C]$$

Características:

- Cada pixel armazena informação de cor
- Representada por valores numéricos
- Pode ter um ou mais canais de cor

Exemplo de matriz 3x3:

120	85	200
45	180	90
255	60	150

Valores de 0 (preto) a 255 (branco)

Pixel: A Menor Unidade

O **pixel** (*Picture Element*) é a menor unidade de uma imagem digital.

Características do Pixel:

- Representa a intensidade de cor em um ponto
- Pode ter um valor (tons de cinza)
- Pode ter múltiplos valores (imagem colorida)
- Identificado por coordenadas (x, y)

Valores típicos:

Tipo	Valores
Binário	0 ou 255
Cinza	0 a 255
RGB	(R, G, B)

Importante: Quanto mais pixels, maior o detalhamento da imagem!

Resolução Espacial

A **resolução espacial** está relacionada com a quantidade de pixels usados para representar a imagem.

$$\text{Total de Pixels} = \text{Largura} \times \text{Altura}$$

Resolução	Total de Pixels	Uso Comum
640 × 480	307.200	VGA
1280 × 720	921.600	HD
1920 × 1080	2.073.600 (~2MP)	Full HD
3840 × 2160	8.294.400 (~8MP)	4K

Maior resolução = Mais detalhes = Maior tamanho de arquivo

Resolução de Bit (Quantização)

A **resolução de bit** corresponde à quantidade de tons de cinza que podem ser atribuídos para cada pixel.

$$\text{Níveis} = 2^{\text{bits}}$$

Bits	Níveis	Descrição
1 bit	2	Preto e branco (binário)
4 bits	16	16 tons de cinza
8 bits	256	256 tons de cinza (padrão)
24 bits	16.777.216	RGB (8 bits por canal)

TIPOS DE IMAGENS DIGITAIS

Imagens Binárias

Cada pixel possui apenas **2 valores**: 0 (preto) ou 255 (branco).

Características:

- **1 bit** por pixel
- Apenas preto e branco
- Usadas para segmentação
- Detecção de formas geométricas
- Menor tamanho de arquivo

| Aplicações: Documentos escaneados, OCR, detecção de bordas.

Representação em Python:

```
# Pixel preto  
pixel = 0  
  
# Pixel branco  
pixel = 255
```

Imagens em Tons de Cinza

Cada pixel possui valores de 0 (preto) a 255 (branco).

Características:

- 8 bits por pixel
- 256 níveis de cinza
- Sem informação de cor
- Mais leves para processar
- Representa intensidade luminosa

Representação em Python:

```
# Pixel preto  
pixel = 0  
  
# Cinza médio  
pixel = 127  
  
# Pixel branco  
pixel = 255
```

Vantagem: Processamento mais rápido que imagens coloridas!

Imagens Coloridas (RGB)

Cada pixel possui **3 canais**: Red (Vermelho), Green (Verde), Blue (Azul).

Características:

- **24 bits** por pixel (8 por canal)
- 2^{24} = **16.777.216 cores**
- Cada canal é como uma imagem em tons de cinza
- Estrutura: **Matriz**[L × C × 3]

Exemplos de cores em Python:

```
# Vermelho puro
pixel = [255, 0, 0]

# Verde puro
pixel = [0, 255, 0]

# Azul puro
pixel = [0, 0, 255]

# Branco
pixel = [255, 255, 255]
```

Convertendo Entre Tipos de Imagem

Código :

```
import cv2

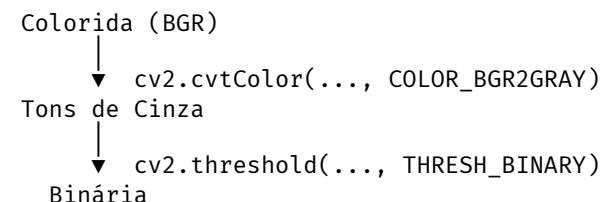
# Carrega como colorida (BGR)
img_bgr = cv2.imread("img/wiki.jpg")

# Colorida → Tons de Cinza
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

# Tons de Cinza → Binária (limiar = 127)
_, img_bin = cv2.threshold(
    img_gray, 127, 255, cv2.THRESH_BINARY
)

# Carrega diretamente em escala de cinza
img_gray2 = cv2.imread("img/wiki.jpg",
                       cv2.IMREAD_GRAYSCALE)
```

Fluxo de conversão:



Shapes resultantes (img.shape):

Tipo	Shape
Colorida	(H, W, 3)
Tons de cinza	(H, W)
Binária	(H, W)

A conversão direta de **colorida para binária** não existe em OpenCV — passe sempre pela escala de cinza.

HISTOGRAMA DE IMAGENS

☰ O que é um Histograma?

O **histograma de cores** é a distribuição de frequência dos níveis de cinza em relação ao número de pixels.

Eixos do histograma:

- **Eixo X:** Níveis de intensidade (0 a 255)
- **Eixo Y:** Quantidade de pixels

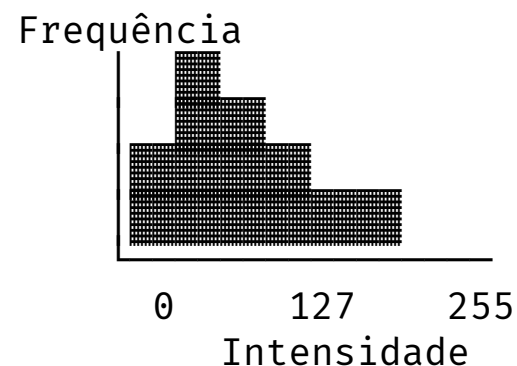
Informações fornecidas:

- Qualidade da imagem
- Nível de luminosidade
- Nível de contraste

Por que histograma e não gráfico de barras?

- **Gráfico de barras:** dados categóricos, sem ordem, barras separadas
- **Histograma:** dados numéricos de uma escala contínua discretizada, barras adjacentes e ordenadas
- Os valores 0–255 são uma **quantização** da intensidade luminosa, grandeza física contínua

Representação visual:



Barras **adjacentes** e **ordenadas**: reflexo da continuidade da escala de intensidade.

Histograma à Mão: Imagem 3x3 (1/2)

Imagem (tons de cinza)

	Col 1	Col 2	Col 3
Lin 1	50	100	150
Lin 2	50	200	100
Lin 3	100	150	50

Total de pixels: $N = 9$

Passo 1: contar cada intensidade

Intensidade	Contagem	Frequência
50	(1,1), (2,1), (3,3)	3
100	(1,2), (2,3), (3,1)	3
150	(1,3), (3,2)	2
200	(2,2)	1

Cada célula da tabela vira uma barra no histograma.

Histograma à Mão: Imagem 3x3 (2/2)

Passo 2: PMF e CDF

r_k	n_k	$p(r_k) = \frac{n_k}{N}$	$CDF(k)$
50	3	$3/9 = 0,33$	0,33
100	3	$3/9 = 0,33$	0,67
150	2	$2/9 = 0,22$	0,89
200	1	$1/9 = 0,11$	1,00

| $CDF(k)$ = soma acumulada das probabilidades até k .

Passo 3: Desenhar o Histograma

```

import numpy as np
import matplotlib.pyplot as plt

img = np.array([[50, 100, 150],
                [50, 200, 100],
                [100, 150, 50]], dtype=np.uint8)

intensidades = [50, 100, 150, 200]
frequencias = [3, 3, 2, 1]

fig, axes = plt.subplots(1, 3, figsize=(11, 3.5))
fig.suptitle("Histograma à Mão: Imagem 3×3", fontweight='bold')

axes[0].imshow(img, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Imagem 3×3")
for i in range(3):
    for j in range(3):
        axes[0].text(j, i, str(img[i, j]), ha='center', va='center',
                     color='red', fontsize=12, fontweight='bold')

axes[1].bar(intensidades, frequencias, width=18, color='#1565c0')
axes[1].set_xticks(intensidades)
axes[1].set_xlabel("Intensidade"); axes[1].set_ylabel("Frequência")
axes[1].set_title("Histograma"); axes[1].set_xlim(0, 256)

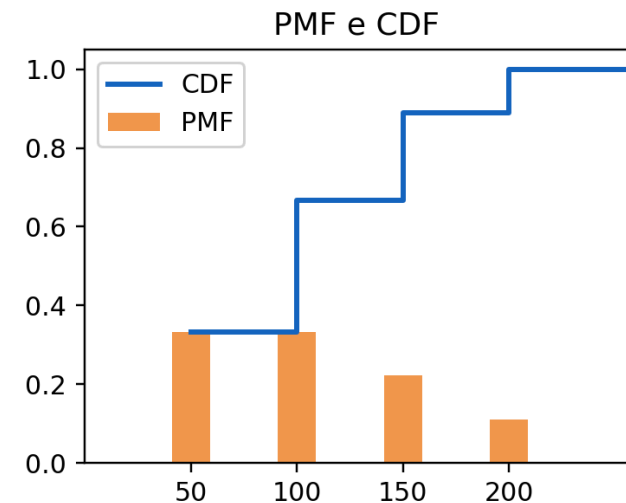
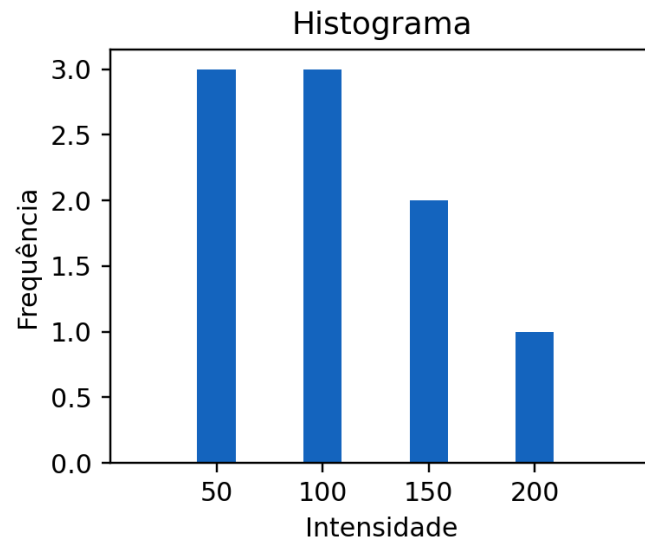
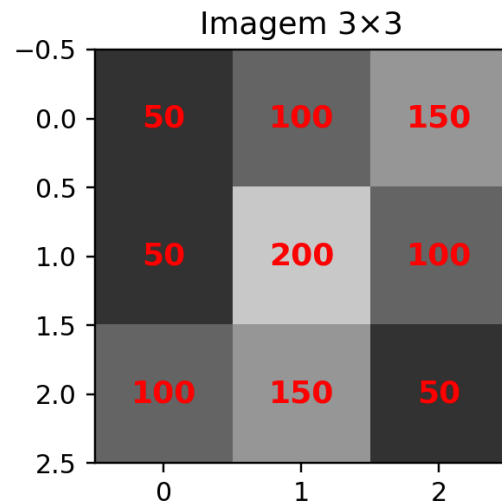
pmf = np.array(frequencias) / 9
cdf = np.cumsum(pmf)
axes[2].bar(intensidades, pmf, width=18, color='#ef6c00', alpha=0.7, label='PMF')
axes[2].step(intensidades + [256], list(cdf) + [cdf[-1]],
             where='post', color='#1565c0', linewidth=2, label='CDF')
axes[2].set_xticks(intensidades); axes[2].set_xlim(0, 256)
axes[2].set_title("PMF e CDF"); axes[2].legend(loc='upper left')

plt.tight_layout(); plt.show()

```

</> Histograma 3x3 com Python

Histograma à Mão: Imagem 3x3



ESTATÍSTICA E HISTOGRAMAS

Leitura Recomendada

Antes de prosseguir, revise os seguintes conceitos no seu livro de [Estatística Básica](#), como por exemplo [Morettin e Bussab \(Estatística Básica, Saraiva\)](#):

- [Variável aleatória](#): definição, tipos (discreta e contínua) e exemplos
- [Histograma](#): construção, interpretação e relação com frequência relativa
- [Função de Probabilidade \(PMF\)](#): definição e propriedades para variáveis discretas
- [Função de Distribuição Acumulada \(CDF\)](#): definição, propriedades e leitura do gráfico

Esses conceitos são a base para compreender como o histograma de uma imagem pode ser interpretado estatisticamente e por que a CDF é usada na equalização.

Histograma como Distribuição Estatística

O histograma de uma imagem é, na verdade, uma **distribuição de frequências**, podendo ser interpretado estatisticamente.

Conceito	Significado
Experimento	Escolher um pixel aleatoriamente
Resultado	Intensidade do pixel (0–255)
Frequência	Quantos pixels têm aquela intensidade
Probabilidade	Frequência relativa ao total de pixels

Ao **normalizar** o histograma pelo número total de pixels N , obtemos a **probabilidade** de um pixel ter cada nível de intensidade.

$$p(r_k) = \frac{n_k}{N}$$

onde:

- r_k = nível de intensidade k , com $k = 0, 1, \dots, 255$
- n_k = número de pixels com intensidade r_k
- N = total de pixels da imagem

Função de Probabilidade (PMF)

A **Função de Massa de Probabilidade** (*PMF*, *Probability Mass Function*) para variáveis **discretas** atribui uma probabilidade a cada valor possível.

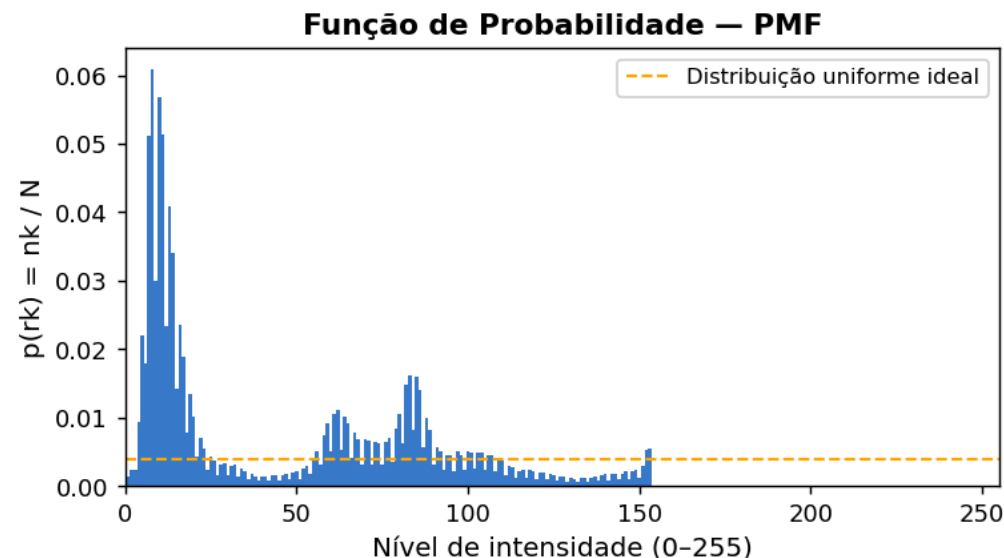
Para uma imagem com N pixels:

$$p(r_k) = \frac{n_k}{N}, \quad k = 0, 1, \dots, 255$$

Propriedades:

- $0 \leq p(r_k) \leq 1$
- $\sum_{k=0}^{255} p(r_k) = 1$

Cada barra do histograma normalizado representa $p(r_k)$.



Função de Distribuição Acumulada (CDF)

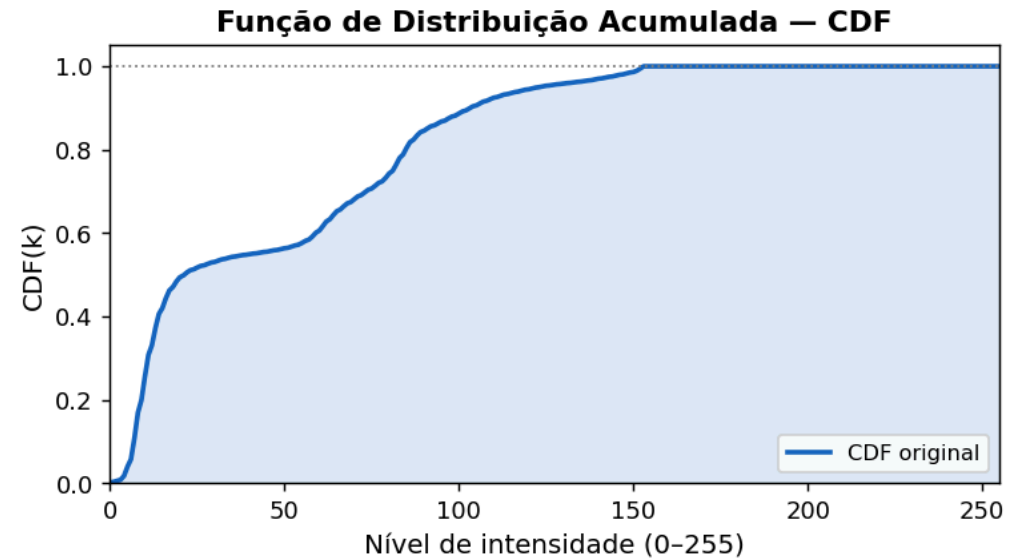
A **CDF** (*Cumulative Distribution Function*) acumula as probabilidades até um dado nível de intensidade:

$$CDF(k) = \sum_{j=0}^k p(r_j) = \sum_{j=0}^k \frac{n_j}{N}$$

Propriedades:

- Sempre **crescente** (nunca diminui)
- $CDF(0) \geq 0$ e $CDF(255) = 1$
- Representa a probabilidade acumulada de um pixel ter intensidade **até** k

No código: `cdf = hist.cumsum()`



↔ Por que usar Probabilidade na Equalização?

O objetivo da equalização é obter uma **distribuição uniforme** de intensidades, ou seja, uma CDF linear.

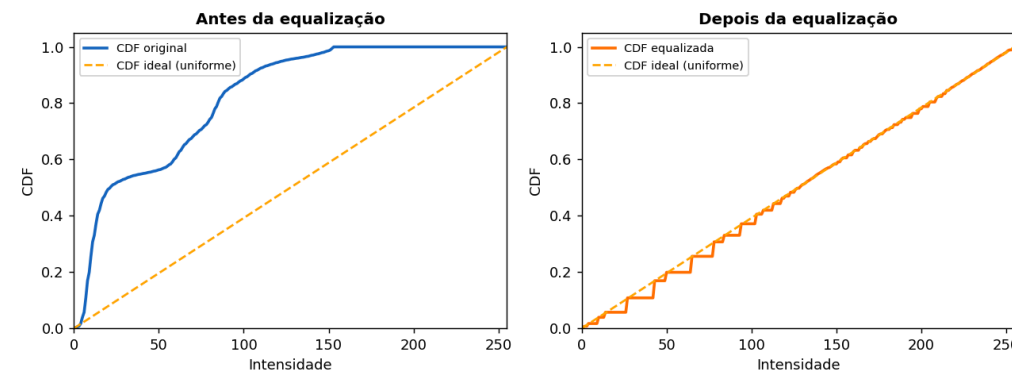
A **transformação de equalização** é definida como:

$$s_k = (L - 1) \cdot CDF(r_k)$$

onde $L = 256$ (número de níveis) e s_k é o novo valor do pixel.

Intuição:

- Se a CDF fosse **perfeitamente linear** → distribuição uniforme
- A transformação "estica" as intensidades concentradas para preencher toda a faixa 0–255



A CDF equalizada se aproxima da linha ideal (uniforme)

√x De onde vem a fórmula $s_k = (L - 1) \cdot CDF(r_k)$?

Objetivo: transformar a distribuição original em uma distribuição **uniforme**.

Uma distribuição uniforme tem CDF perfeitamente linear:

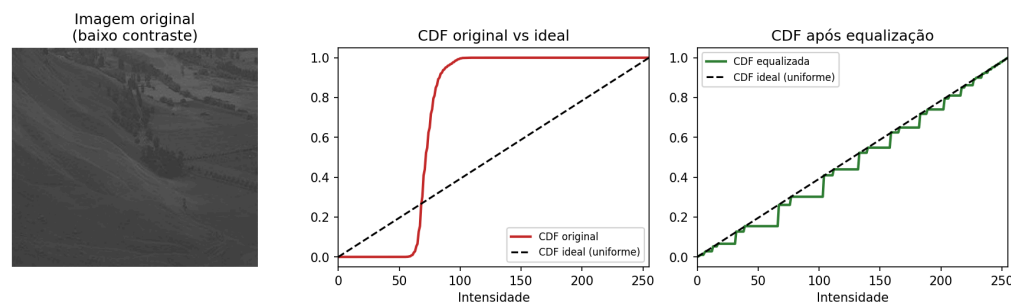
$$CDF_{ideal}(x) = \frac{x}{L-1}$$

Queremos encontrar s_k tal que a CDF da imagem transformada seja linear. Aplicando a transformação inversa:

$$CDF(r_k) = \frac{s_k}{L-1} \implies s_k = (L-1) \cdot CDF(r_k)$$

Por que $(L-1)$? É o valor máximo de intensidade (255 para 8 bits). Multiplicar pela CDF (que vai de 0 a 1) escala o resultado para o intervalo $[0, 255]$.

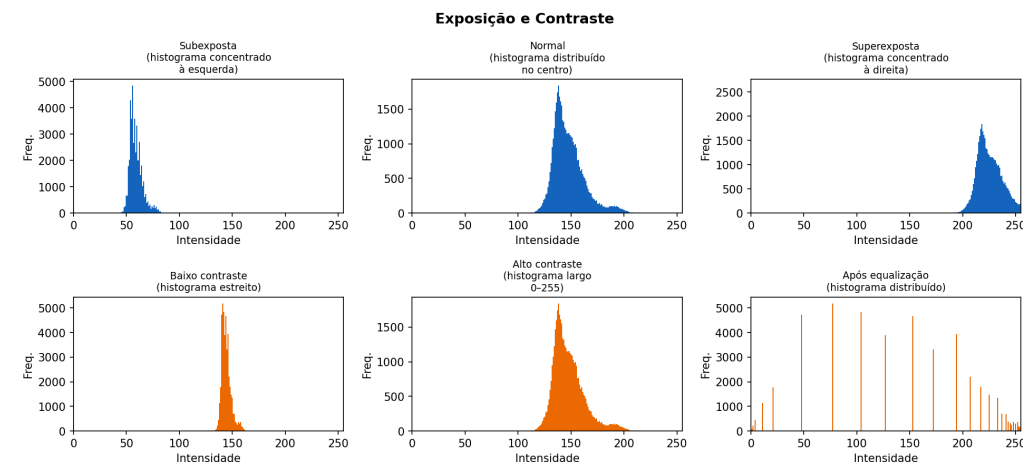
Por que a CDF? Ela acumula probabilidade — pixels com intensidades muito frequentes terão CDF crescendo rapidamente, sendo "espalhados" por mais níveis de saída.



Antes: CDF original afastada do ideal. Depois: CDF equalizada aproxima-se da linha linear.

Interpretação: Exposição e Contraste

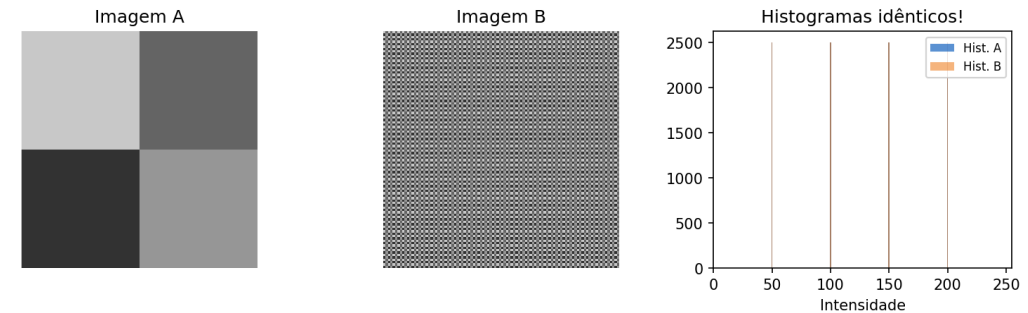
Tipo	Histograma
Subexposta	Concentrado à esquerda
Normal	Distribuído no centro
Superexposta	Concentrado à direita
Baixo contraste	Estreito
Alto contraste	Largo (0-255)
Equalizada	Distribuído uniformemente



⚠ Limitações do Histograma

O histograma **não preserva informação espacial** da imagem.

- Imagens completamente diferentes podem ter **histogramas idênticos**
- Pixels de diferentes regiões com mesmos valores produzem a mesma distribuição
- Não é possível reconstruir a imagem a partir do histograma



Consequências práticas:

- Não use o histograma como única métrica de qualidade
- Equalização pode **amplificar ruído** em regiões uniformes
- Em imagens muito escuras ou muito claras, o efeito pode ser exagerado

Duas imagens distintas com histogramas idênticos

EQUALIZAÇÃO DE HISTOGRAMA

O que é Equalização de Histograma?

É uma técnica de **realce de imagem** que redistribui os níveis de cinza para melhorar o contraste.

Objetivo:

Fazer com que os pixels ocupem **toda a faixa disponível** (0 a 255).

Quando usar:

- Imagens com baixo contraste
- Ambientes com iluminação inadequada
- Objetos de interesse pouco nítidos

Etapas:

1. Calcular o histograma original
2. Calcular distribuição acumulada (CDF)
3. Mapear valores antigos para novos
4. Gerar nova imagem equalizada

⚠ Equalização em Imagens Coloridas

❗ Por que não equalizar direto no RGB?

No modelo RGB, **cor e brilho estão misturados** nos três canais. Equalizar cada canal separadamente altera proporções diferentes de R, G e B, gerando **dominâncias de cor artificiais**.

Exemplo: um pixel (200, 100, 50) equalizado canal a canal pode virar (230, 180, 210), mudando completamente a cor percebida.

✅ Por que o HSV resolve?

O HSV **separa cor de brilho**. Ao equalizar **somente o canal V**, ajustamos o contraste sem tocar na cor. H e S permanecem intactos, preservando a identidade visual da imagem.

Etapas:

1. Converter **RGB** → **HSV**
2. Equalizar apenas o canal **V**
3. Converter de volta **HSV** → **RGB**

Canal	Significado	Equalizar?
H (Hue)	Matiz/Cor	×
S (Saturation)	Saturação	×
V (Value)	Intensidade/Brilho	✓

Regra geral: **equalize apenas a dimensão de brilho**, nunca as de cor.

BIBLIOTECAS UTILIZADAS



Instalação: `pip install numpy`

Contexto nesta disciplina: Estrutura base para representar imagens como matrizes numéricas, calcular histogramas, CDF e aplicar transformações matemáticas.

Função	Descrição
<code>np.array()</code>	Converte dados em matriz
<code>np.zeros()</code> / <code>np.ones()</code>	Cria matrizes de zeros ou uns
<code>np.histogram()</code>	Calcula o histograma
<code>np.cumsum()</code>	Soma acumulada (CDF)
<code>np.uint8()</code>	Converte para 8 bits (0–255)
<code>img.flatten()</code>	Achata matriz para 1D
<code>img.shape</code>	Retorna dimensões da imagem



Instalação: `pip install opencv-python`

Contexto nesta disciplina: Principal biblioteca para leitura, escrita, conversão de espaços de cor e processamento de imagens (equalização, filtros, segmentação).

Função	Descrição
<code>cv2.imread()</code>	Carrega imagem do disco
<code>cv2.imwrite()</code>	Salva imagem no disco
<code>cv2.cvtColor()</code>	Converte espaço de cor
<code>cv2.equalizeHist()</code>	Equalização de histograma
<code>cv2.split() / merge()</code>	Separa/junta canais
<code>cv2.resize()</code>	Redimensiona imagem
<code>cv2.GaussianBlur()</code>	Filtro de suavização



Instalação: `pip install matplotlib`

Contexto nesta disciplina: Visualização de imagens, histogramas e gráficos de CDF. Permite comparar resultados antes e depois do processamento.

Função	Descrição
<code>plt.imshow()</code>	Exibe imagem
<code>plt.hist()</code>	Plota histograma
<code>plt.plot()</code>	Plota linha (ex.: CDF)
<code>plt.subplots()</code>	Cria grade de subplots
<code>plt.bar()</code>	Gráfico de barras
<code>plt.show()</code>	Renderiza o gráfico
<code>plt.savefig()</code>	Salva figura em arquivo

PRÁTICA COM PYTHON E OPENCV

Documentação do OpenCV

Ao longo das aulas práticas, os exemplos de código serão baseados na [documentação oficial do OpenCV](#).

Por que usar a documentação:

- Referência completa de todas as funções
- Exemplos prontos e testados
- Atualizada a cada nova versão da biblioteca
- Contém tutoriais por tema (histograma, filtros, segmentação...)

Como acessar:

- Site oficial: docs.opencv.org
- Navegue por módulos ou use a busca
- Prefira os tutoriais em [Python](#)
- Cada função tem assinatura, parâmetros e exemplos

⚠ Atenção: OpenCV usa BGR, não RGB!

O OpenCV carrega imagens na ordem **BGR** (Blue, Green, Red), ao contrário do padrão **RGB** usado pelo matplotlib e pela maioria das bibliotecas.

Consequência: exibir uma imagem colorida carregada com `cv2.imread()` diretamente no matplotlib inverte os canais vermelho e azul, distorcendo as cores.

```
img_bgr = cv2.imread("imagem.jpg")      # BG
img_rgb = cv2.cvtColor(img_bgr,          # co
                      cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)    # correto
plt.imshow(img_bgr)    # cores distorcidas!
```

Para imagens em **escala de cinza** este problema não existe, pois há apenas um canal.

cv2.imread() sem conversão
(BGR interpretado como RGB → cores erradas)



Após cv2.COLOR_BGR2RGB
(cores corretas)





Carregando Imagens e Gerando Histograma

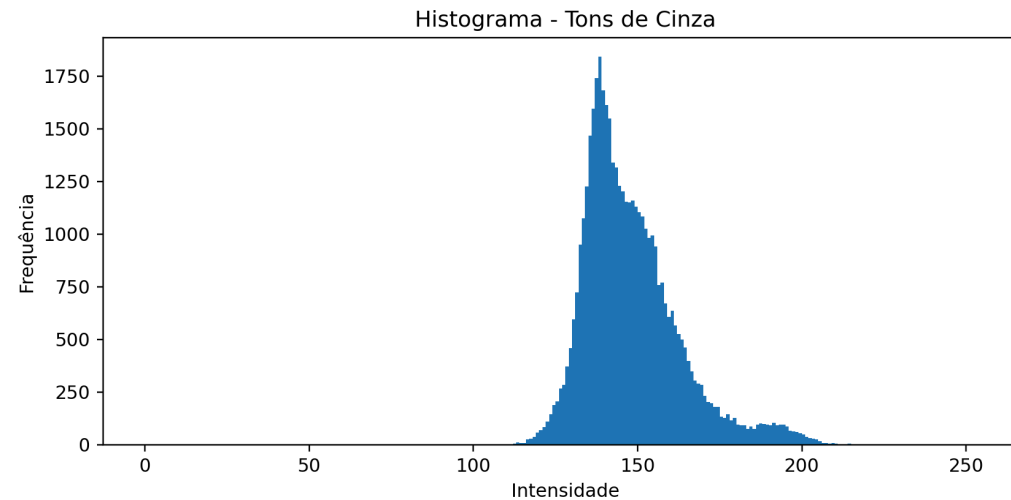
```
import cv2
from matplotlib import pyplot as plt

# Carregar imagem em tons de cinza
imagem = cv2.imread("img/wiki.jpg", 0)

# Plotar histograma
plt.hist(imagem.ravel(), 256, [0, 256])
plt.title("Histograma - Tons de Cinza")
plt.xlabel("Intensidade")
plt.ylabel("Frequência")
plt.show()
```



Carregando Imagens e Gerando Histograma



Histograma + CDF com NumPy

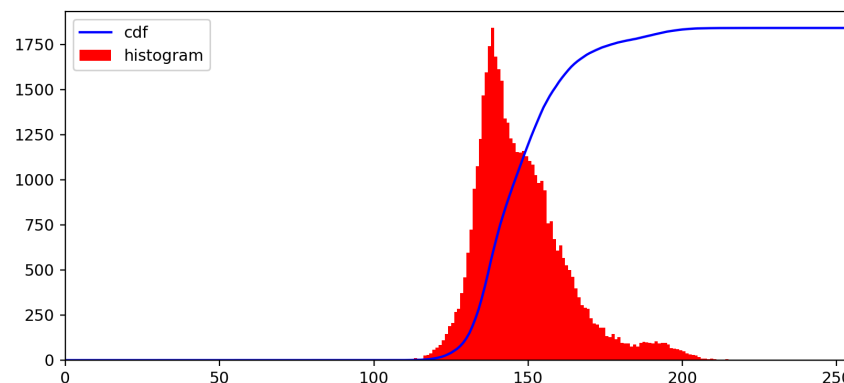
```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread("img/wiki.jpg",
                cv.IMREAD_GRAYSCALE)
hist, bins = np.histogram(
    img.flatten(), 256, [0, 256])

cdf = hist.cumsum()
cdf_normalized = (cdf *
                 float(hist.max()) / cdf.max())

plt.plot(cdf_normalized, color='b')
plt.hist(img.flatten(),
         256, [0, 256], color='r')
plt.xlim([0, 256])
plt.legend(('cdf', 'histogram'),
          loc='upper left')
plt.show()
```

(0.0, 256.0)



Equalização com OpenCV

```
import cv2
from matplotlib import pyplot as plt

# Tons de cinza
imagem_original = cv2.imread("img/wiki.jpg", 0)
imagem_equalizada = cv2.equalizeHist(imagem_original)

# Imagem colorida (via HSV)
imagem_bgr = cv2.imread("img/wiki.jpg")
imagem_hsv = cv2.cvtColor(imagem_bgr, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(imagem_hsv)
v_equalizado = cv2.equalizeHist(v)
imagem_hsv_eq = cv2.merge([h, s, v_equalizado])
imagem_eq = cv2.cvtColor(imagem_hsv_eq, cv2.COLOR_HSV2BGR)

fig, axes = plt.subplots(1, 2, figsize=(8, 3))
axes[0].imshow(cv2.cvtColor(imagem_bgr, cv2.COLOR_BGR2RGB))
axes[0].set_title("Original"); axes[0].axis('off')
axes[1].imshow(cv2.cvtColor(imagem_eq, cv2.COLOR_BGR2RGB))
axes[1].set_title("Equalizada (HSV)"); axes[1].axis('off')
plt.tight_layout(); plt.show()
```

Original



Equalizada (HSV)



</> Comparação: Original vs. Equalizada

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img      = cv.imread("img/wiki.jpg", cv.IMREAD_GRAYSCALE)
img_eq   = cv.equalizeHist(img)

fig, axes = plt.subplots(2, 2, figsize=(11, 8))
fig.suptitle("Equalização de Histograma", fontweight='bold')

axes[0, 0].imshow(img,      cmap='gray'); axes[0, 0].set_title("Original");   axes[0, 0].axis('off')
axes[0, 1].imshow(img_eq,  cmap='gray'); axes[0, 1].set_title("Equalizada"); axes[0, 1].axis('off')

for ax, data, title in zip(axes[1], [img, img_eq], ["Original", "Equalizada"]):
    h, _ = np.histogram(data.flatten(), 256, [0, 256])
    cdf_ = h.cumsum(); cdf_n = cdf * float(h.max()) / cdf.max()
    ax.bar(range(256), h, color='red', alpha=0.6, label='Histograma')
    ax.plot(cdf_n, color='blue', linewidth=1.5, label='CDF')
    ax.set_xlim([0, 256]); ax.set_title(f"Histograma + CDF ({title})")
    ax.legend(loc='upper left')

plt.tight_layout(); plt.show()
```

</> Comparação: Original vs. Equalizada

Equalização de Histograma

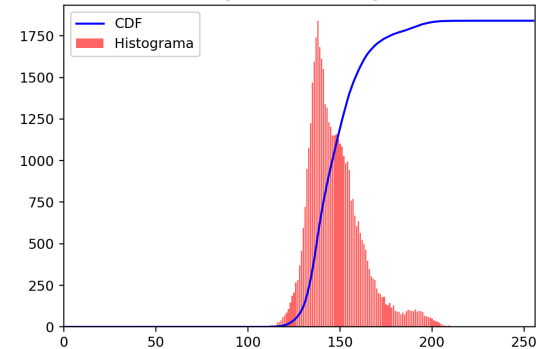
Original



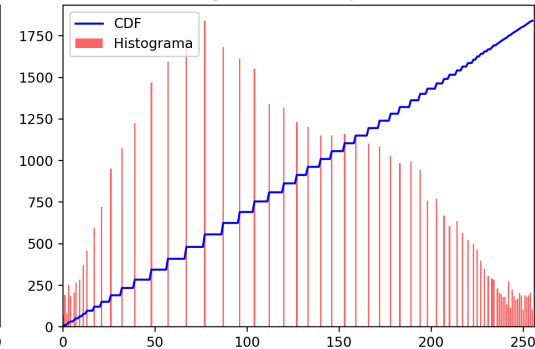
Equalizada



Histograma + CDF (Original)



Histograma + CDF (Equalizada)



DESAFIO

Desafio: Função de Equalização

O que fazer:

- Implemente a função `equalizar_imagem(caminho)`
- Recebe o `caminho` de uma imagem como argumento
- **Verifica** se a imagem é colorida ou em escala de cinza:
 - Se **colorida**: converte para HSV e equaliza apenas o canal V
 - Se **em escala de cinza**: aplica a equalização diretamente
- Retorna **três elementos**:
 - A imagem equalizada
 - O histograma (256 bins)
 - A CDF normalizada

Entrega:

- Arquivo `.py` ou `.ipynb`
- Submeta pelo **Blackboard** até a data indicada
- **Bônus**: plote os 4 painéis (original, equalizada, histograma e CDF)

Bibliotecas e funções permitidas:

cv2 (OpenCV)

- `cv2.imread(caminho)`: carrega a imagem
- `img.shape`: verificar número de canais (`len == 3` → colorida)
- `cv2.cvtColor(img, cv2.COLOR_BGR2HSV)`: converte para HSV
- `cv2.equalizeHist(canal)`: equaliza um canal
- `cv2.cvtColor(img, cv2.COLOR_HSV2BGR)`: converte de volta

numpy

- `np.histogram(img.flatten(), 256, [0,256])`: histograma
- `hist.cumsum() / cdf / cdf.max()`: CDF normalizada

matplotlib

- `plt.imshow(), plt.bar(), plt.plot(), plt.subplots()`

Resumo da Aula

Conceito	Definição
Imagem Digital	Matriz bidimensional de pixels
Pixel	Menor unidade, armazena intensidade de cor
Resolução Espacial	Quantidade de pixels (linhas × colunas)
Resolução de Bit	Níveis de cinza (8 bits = 256 níveis)
Histograma	Distribuição de frequência dos níveis de cinza
Equalização	Redistribui níveis para melhorar contraste

Referências Bibliográficas

1. GONZALEZ, R. C.; WOODS, R. E. [Processamento de imagens digitais](#). São Paulo: Blucher, 2000.
2. BARELLI, F. [Introdução à Visão Computacional: Uma abordagem prática com Python e OpenCV](#). São Paulo: Casa do Código, 2019.
3. JAIN, A. K. [Fundamentals of digital image processing](#). Prentice-Hall, 1989.

EXERCÍCIOS



Exercícios - Parte I: Conceitos e Interpretação

1. Uma imagem de 800×600 pixels com 8 bits de profundidade apresenta $n_k = 1440$ pixels com intensidade $r_k = 128$. Calcule $p(r_k)$ e interprete o resultado.

2. Considere a imagem 3×3 abaixo. Calcule manualmente o histograma, a PMF e a CDF para cada nível de intensidade presente:

$$\begin{bmatrix} 80 & 200 & 80 \\ 150 & 80 & 200 \\ 200 & 150 & 150 \end{bmatrix}$$

3. A partir da CDF calculada no exercício anterior, aplique a fórmula de equalização $s_k = (L - 1) \cdot CDF(r_k)$ com $L = 256$ e determine os novos níveis de intensidade de cada pixel.

4. Explique com suas palavras por que o histograma de uma imagem equalizada tende a ser mais uniforme que o original. Em que condições a equalização pode piorar a qualidade visual da imagem?

5. Duas imagens distintas possuem histogramas idênticos. O que isso implica sobre a relação entre histograma e conteúdo espacial da imagem?

6. Uma imagem colorida é carregada com `cv2.imread()` e exibida diretamente com `plt.imshow()` sem nenhuma conversão. O que acontece com as cores e por quê?



Exercícios - Parte II: Programação

7. Escreva um código Python que:

- Carregue `img/wiki.jpg` em tons de cinza
- Calcule o histograma com `np.histogram()` (256 bins)
- Calcule a CDF normalizada
- Plote histograma e CDF no mesmo gráfico

8. Modifique o código do exercício anterior para que, ao invés de usar `cv2.equalizeHist()`, a equalização seja feita **manualmente** aplicando $s_k = (L - 1) \cdot CDF(r_k)$ pixel a pixel com NumPy. Compare o resultado com `cv2.equalizeHist()`.

9. Carregue `img/wiki.jpg` como imagem colorida e aplique a equalização corretamente via espaço HSV. Exiba lado a lado: imagem original, equalizada via HSV e equalizada diretamente em cada canal RGB. Discuta as diferenças visuais.

10. Dada a imagem 3×3 abaixo representada como array NumPy, use Python para plotar a imagem, seu histograma e a PMF+CDF em um painel de 3 subplots:

```
img = np.array([[80, 200, 80],  
                [150, 80, 200],  
                [200, 150, 150]], dtype=np.u
```

11. (Desafio extra) Implemente do zero, sem usar `cv2.equalizeHist()`, uma função que receba qualquer imagem em escala de cinza e retorne a versão equalizada. Valide comparando com o resultado da função do OpenCV usando `np.allclose()`.

PRÓXIMA AULA

Realce de imagens no domínio do espaço

Filtro da Média | Filtro Gaussiano | Filtro da Mediana

OBRIGADO!

 jjairo@up.edu.br

 **Universidade Positivo**